

```

import cv2
import numpy as np
import os
from scipy.ndimage import generic_filter
import matplotlib.pyplot as plt
# === Load RGB image ===
img_bgr = cv2.imread("Conor.jpg")
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Create output folders
os.makedirs("noisy_images_rgb", exist_ok=True)
os.makedirs("filtered_images_rgb", exist_ok=True)

# === Noise functions applied per channel ===
def add_noise_rgb(image, method):
    noisy = image.copy().astype(np.float32)

    for c in range(3): # for each channel
        if method == 'gaussian':
            noise = np.random.normal(0, 25, image[:, :, c].shape)
            noisy[:, :, c] += noise
        elif method == 'uniform':
            noise = np.random.uniform(50, 100, image[:, :, c].shape)
            noisy[:, :, c] += noise
        elif method == 'rayleigh':
            noise = np.random.rayleigh(20, image[:, :, c].shape)
            noisy[:, :, c] += noise
        elif method == 'gamma':
            noise = np.random.gamma(2, 20, image[:, :, c].shape)
            noisy[:, :, c] += noise
        elif method == 'exponential':
            noise = np.random.exponential(20, image[:, :, c].shape)
            noisy[:, :, c] += noise
        elif method == 'salt_and_pepper':
            # Apply salt and pepper per channel
            noisy[:, :, c] = impulse_noise(image[:, :, c])
        else:
            pass

    return np.clip(noisy, 0, 255).astype(np.uint8)

# === Salt and pepper noise ===
def impulse_noise(channel, noise_ratio=0.05):
    noisy = channel.copy()
    h, w = noisy.shape
    num_salt = int(noise_ratio * h * w / 2)
    num_pepper = int(noise_ratio * h * w / 2)

    # Salt
    coords = (np.random.randint(0, h, num_salt), np.random.randint(0,

```

```

w, num_salt))
    noisy[coords] = 255

    # Pepper
    coords = (np.random.randint(0, h, num_pepper),
np.random.randint(0, w, num_pepper))
    noisy[coords] = 0

    return noisy

# === Apply noise and save RGB ===
noise_methods = ['gaussian', 'uniform', 'rayleigh', 'gamma',
'exponential', 'salt_and_pepper']
noisy_rgb_images = {}

for method in noise_methods:
    noisy_rgb = add_noise_rgb(img_rgb, method)
    cv2.imwrite(f"noisy_images_rgb/{method}.jpg",
cv2.cvtColor(noisy_rgb, cv2.COLOR_RGB2BGR))
    noisy_rgb_images[method] = noisy_rgb

# === Load Original Image in RGB ===
original_bgr = cv2.imread('conor.jpg')
original_rgb = cv2.cvtColor(original_bgr, cv2.COLOR_BGR2RGB)

# Noise Definitions
noise_methods = ['gaussian', 'uniform', 'rayleigh', 'gamma',
'exponential', 'salt_and_pepper']

# === Load Noisy Images ===
noisy_images = []
for method in noise_methods:
    path = os.path.join('noisy_images_rgb', f"{method}.jpg")
    img = cv2.imread(path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    noisy_images.append((method.replace('_', ' ').capitalize(),
img_rgb))

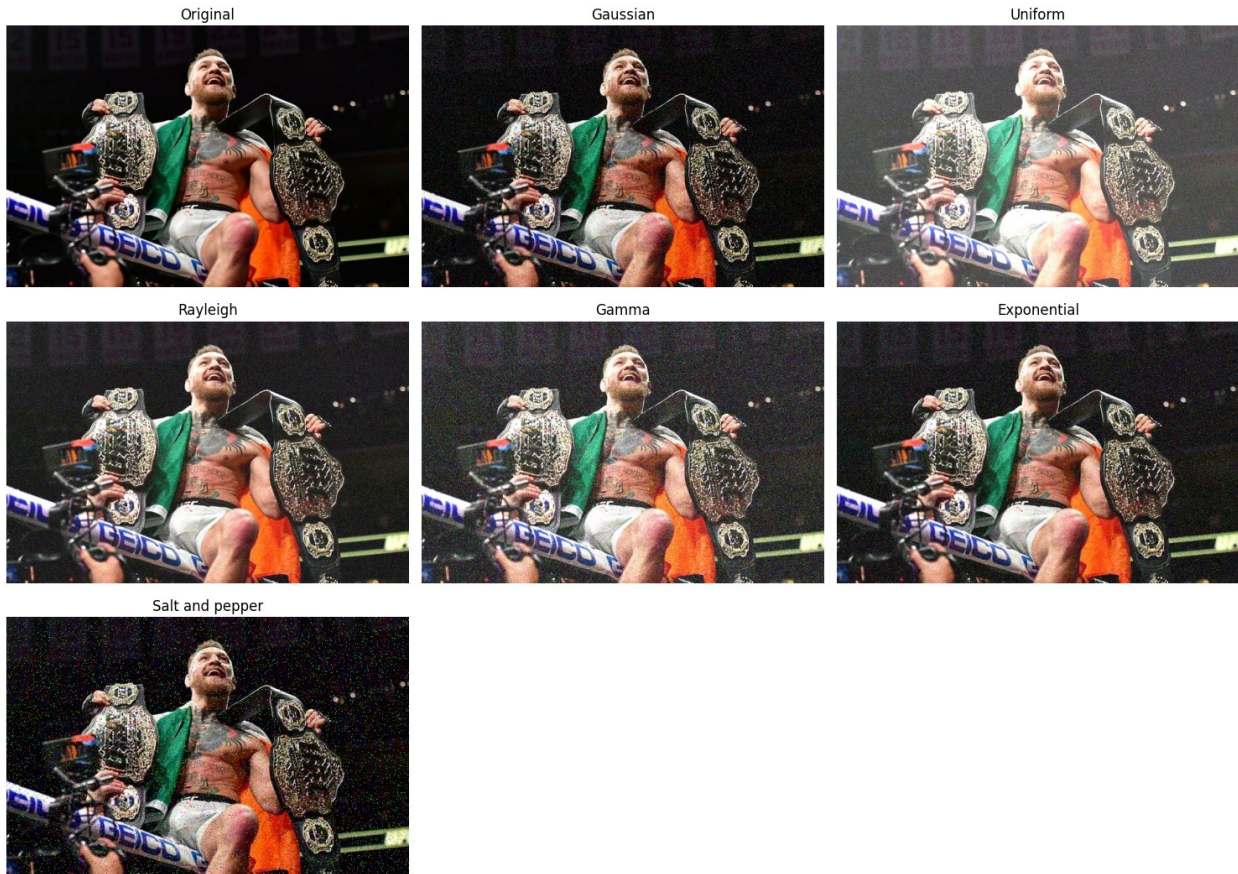
# === DISPLAY FUNCTION ===
def display_images(title, images, cols=3):
    rows = (len(images) + cols - 1) // cols
    plt.figure(figsize=(5 * cols, 4 * rows))
    for i, (name, img) in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(img)
        plt.title(name)
        plt.axis('off')
    plt.suptitle(title, fontsize=20)

```

```
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

```
# === DISPLAY RESULTS ===
display_images("Original + Noisy RGB Images", [('Original',
original_rgb)] + noisy_images)
```

Original + Noisy RGB Images



```
import cv2
import numpy as np
import os
from scipy.ndimage import generic_filter
import matplotlib.pyplot as plt

# === Setup paths ===
img_path = 'conor.jpg'
noisy_dir = 'noisy_images_rgb'
filtered_dir = 'filtered_images_rgb'
os.makedirs(filtered_dir, exist_ok=True)

# === Load original image ===
```

```

img_bgr = cv2.imread(img_path)
if img_bgr is None:
    raise ValueError("Image not found!")

img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# === Load noisy RGB images ===
noise_methods = ['gaussian', 'uniform', 'rayleigh', 'gamma',
'exponential', 'salt_and_pepper']
noisy_rgb_images = {}

for method in noise_methods:
    path = os.path.join(noisy_dir, f"{method}.jpg")
    noisy_bgr = cv2.imread(path)
    if noisy_bgr is not None:
        noisy_rgb_images[method] = cv2.cvtColor(noisy_bgr,
cv2.COLOR_BGR2RGB)

# === Filtering Functions ===
def apply_filter_rgb(img_rgb, filter_func):
    filtered = np.zeros_like(img_rgb)
    for c in range(3):
        filtered[:, :, c] = filter_func(img_rgb[:, :, c])
    return filtered

def arithmetic_mean(img):
    kernel = np.ones((3, 3), dtype=np.float32) / 9
    return cv2.filter2D(img, -1, kernel)

def geometric_mean(img):
    def geo_mean(window):
        window = window[window > 0]
        return np.exp(np.mean(np.log(window))) if len(window) > 0 else
1
    return generic_filter(img.astype(np.float32), geo_mean,
size=3).astype(np.uint8)

def harmonic_mean(img):
    def harm_mean(window):
        window = window[window > 0]
        return len(window) / np.sum(1.0 / window) if len(window) > 0
else 1
    return generic_filter(img.astype(np.float32), harm_mean,
size=3).astype(np.uint8)

def median(img):
    return cv2.medianBlur(img, 3)

def min_filter(img):
    return cv2.erode(img, np.ones((3, 3), np.uint8))

```

```

def max_filter(img):
    return cv2.dilate(img, np.ones((3, 3), np.uint8))

def midpoint(img):
    return ((min_filter(img).astype(np.uint16) +
max_filter(img).astype(np.uint16)) // 2).astype(np.uint8)

filter_funcs = {
    '1': ('arithmetic_mean', arithmetic_mean),
    '2': ('geometric_mean', geometric_mean),
    '3': ('harmonic_mean', harmonic_mean),
    '4': ('median', median),
    '5': ('min', min_filter),
    '6': ('max', max_filter),
    '7': ('midpoint', midpoint),
}

# === MENU ===
print("==== FILTER APPLICATION MENU ===")
print("Select base image:")
print("0: Original")
for i, method in enumerate(noise_methods, 1):
    print(f"{i}: {method.replace('_', ' ').capitalize()}")

choice = input("Enter your choice (e.g., 0 for original, 2 for
Rayleigh): ").strip()

if choice == '0':
    selected_img = img_rgb
    image_label = 'original'
elif choice.isdigit() and 1 <= int(choice) <= len(noise_methods):
    idx = int(choice) - 1
    image_label = noise_methods[idx]
    selected_img = noisy_rgb_images[image_label]
else:
    raise ValueError("Invalid selection for base image.")

# === FILTER SELECTION ===
print("\nAvailable Filters:")
for key, (name, _) in filter_funcs.items():
    print(f"{key}: {name.replace('_', ' ').capitalize()}")

selected_filters = input("Enter filter numbers to apply (e.g., 1 2 4):
").strip().split()

# === APPLY FILTERS AND DISPLAY ===
results = [('Original/' + image_label.capitalize(), selected_img)]

for f_key in selected_filters:

```

```

        if f_key in filter_funcs:
            fname, f_func = filter_funcs[f_key]
            filtered_rgb = apply_filter_rgb(selected_img, f_func)
            out_path = os.path.join(filtered_dir,
f"{image_label}_{fname}.jpg")
            cv2.imwrite(out_path, cv2.cvtColor(filtered_rgb,
cv2.COLOR_RGB2BGR))
            results.append((fname.replace('_', ' ').capitalize(),
filtered_rgb))
            print(f"Saved: {out_path}")
        else:
            print(f"Invalid filter option: {f_key}")

```

=== DISPLAY RESULTS ===

```

def display_images(title, images, cols=3):
    rows = (len(images) + cols - 1) // cols
    plt.figure(figsize=(5 * cols, 4 * rows))
    for i, (label, img) in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(img)
        plt.title(label)
        plt.axis('off')
    plt.suptitle(title, fontsize=20)
    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()

```

```
display_images("Filtered Results", results)
```

==== FILTER APPLICATION MENU ====

Select base image:

```

0: Original
1: Gaussian
2: Uniform
3: Rayleigh
4: Gamma
5: Exponential
6: Salt and pepper

```

Enter your choice (e.g., 0 for original, 2 for Rayleigh): 0

Available Filters:

```

1: Arithmetic mean
2: Geometric mean
3: Harmonic mean
4: Median
5: Min
6: Max
7: Midpoint

```


Enter filter numbers to apply (e.g., 1 2 4): 1 2 3 4 5 6 7

Saved: filtered_images_rgb\original_arithmetic_mean.jpg

Saved: filtered_images_rgb\original_geometric_mean.jpg

Saved: filtered_images_rgb\original_harmonic_mean.jpg

Saved: filtered_images_rgb\original_median.jpg

Saved: filtered_images_rgb\original_min.jpg

Saved: filtered_images_rgb\original_max.jpg

Saved: filtered_images_rgb\original_midpoint.jpg

Filtered Results

Original/Original



Arithmetic mean



Geometric mean



Harmonic mean



Median



Min



Max



Midpoint

